MicroROM-NPU

A Microprogrammed Neural Inference Accelerator in SystemVerilog

Purpose • Target application • Architecture • Modules • Microprogram flow

MicroROM-NPU

A Microprogrammed Neural Inference Accelerator in SystemVerilog

Design • Implement • Verify • Learn Real Hardware Concepts

A Compact, Educational **NPU** Architecture that Connects Microarchitecture with Neural Computation

1 Purpose of the System

The **MicroROM-NPU** is a small, microprogrammed neural inference accelerator that executes a vector dot product, bias addition and ReLU activation using a MicroROM-controlled datapath. It demonstrates how microarchitecture concepts (microinstructions, sequencing, control store) can be directly applied to a real neural computation workload.

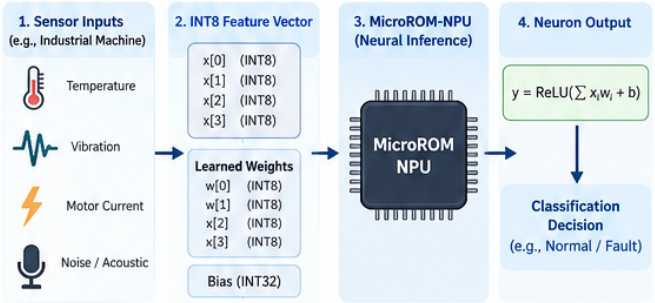
Key Objectives

- Execute a small neural-network inference operation using signed INT8 activations and weights.
- Use a MicroROM to store the microprogram.
- Use a microsequencer and loop counter to process variable-length vectors.
- Implement bias addition, ReLU and output generation.
- Verify numerical correctness against a software reference.
- Measure execution cycles and resource usage.

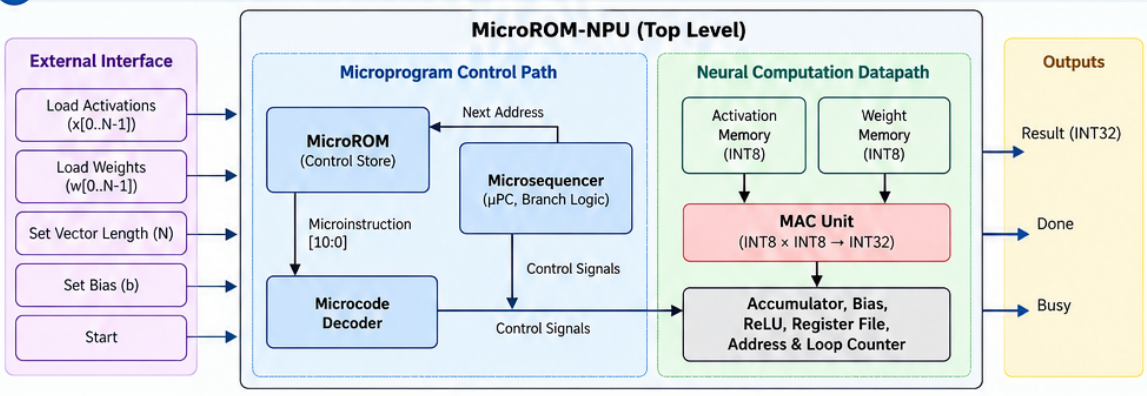
2 Target Application (Real-World Mapping)

Sensor-Based Condition Monitoring (Edge AI)

A compact, single-neuron inference engine can be used in embedded sensor devices (e.g., industrial monitoring, smart IoT nodes) to classify whether a machine is operating normally or showing signs of abnormal behaviour using a few sensor features.



3 Overall System Architecture and Module Overview



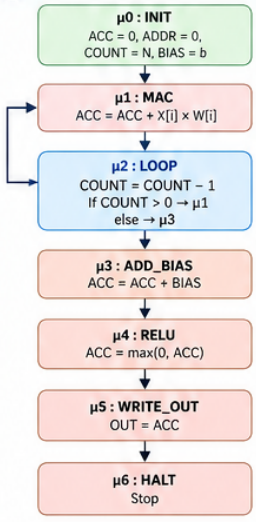
4 Computation Performed

For a vector of length N (e.g., N = 4):

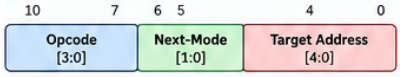
$$y = \mathrm{ReLU}\left(\sum_{i=0}^{N-1} x_i \cdot w_i + b\right)$$

- x_i : INT8 activations (sensor features)
- w_i : INT8 weights (learned parameters)
- b : INT32 bias
- ReLU(z) = max(0, z)
- Sequential MAC executed using a microcode loop

5 Microprogram Flow



6 Microinstruction Format (11 bits)



Opcode (Example)

0	INIT
1	MAC
2	LOOP
3	ADD_BIAS
4	RELU
5	WRITE_OUT
6	HALT

Next-Mode Encoding

00	Sequential (μPC + 1)
01	Unconditional Jump
10	Conditional Loop (if COUNT > 0)
11	Reserved

7 List of Modules in the System

A. Microprogram Control

1.	**microrom.sv**	Control Store (11-bit microinstructions)
2.	**microsequencer.sv**	μPC and branch logic
3.	**microcode_decoder.sv**	Decode fields and generate control signals

B. Neural Computation Datapath

4.	**activation_mem.sv**	Activation buffer (INT8)
5.	**weight_mem.sv**	Weight buffer (INT8)
6.	**mac_unit.sv**	Multiply-Accumulate unit
7.	**npu_datapath.sv**	Accumulator, bias, ReLU, counter, etc.

C. System Integration and Verification

8.	**npu_top.sv**	Top-level integration
9.	**npu_tb.sv**	Testbench and verification

8 Example Use Case (N = 4)

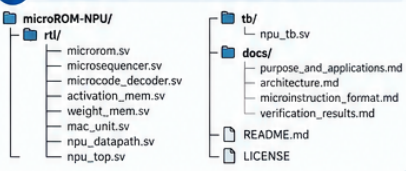
Inputs (INT8)

x = [2, −1, 3, 0]
w = [1, 6, −2, 4]
b = 5 (INT32)

Computation

$\sum x_i w_i = 2\cdot1 + (-1)\cdot6 + 3\cdot(-2) + 0\cdot4$
$= 2 - 6 - 6 + 0 = -10$
$y = \mathrm{ReLU}(-10 + 5) = \mathrm{ReLU}(-5) = 0$

9 Repository Structure (Suggested)



Infographic overview. The detailed specification and implementation notes follow on the next pages.

1. Purpose of the System

MicroROM-NPU is a small, educational, microprogrammed neural-inference accelerator. It executes a vector dot product followed by bias addition and ReLU activation, with a MicroROM controlling the operation sequence. The project demonstrates how microinstructions, a microprogram counter, branch logic, and a reusable datapath can implement a real neural-computation kernel.

Key objectives

• Execute a small inference operation using signed INT8 activations and weights.

• Store the control sequence as microinstructions in a MicroROM.

• Use a microsequencer and loop counter to process variable-length vectors.

• Implement bias addition, ReLU, and output generation.

• Verify numerical correctness against a software reference model.

• Measure execution cycles and, where possible, synthesis/resource estimates.

2. Target Application: Embedded Sensor-Based Condition Monitoring

A suitable small-scale application is an embedded edge device that scores a few sensor-derived features to help distinguish normal operation from a potentially abnormal condition. Example features include temperature, vibration, motor current, and acoustic/noise measurements. These features are scaled into signed INT8 values and processed using learned weights and a bias.

Computation performed

$$y = \mathrm{ReLU}\left(\sum x_i \times w_i + b \right)$$

The output can feed a threshold comparator or a later neural-network layer. The prototype demonstrates the computation architecture; it is not by itself a validated industrial fault detector. A practical system would require a trained model, representative sensor data, suitable feature scaling, and measured accuracy.

Stage	Meaning	Design mapping
Sensor features	Four normalized readings or extracted features	Input interface
INT8 vector	Compact numeric representation of activations	Activation buffer
Learned parameters	Weights and bias from a trained model	Weight buffer and bias register
Neural computation	Dot product, bias addition, ReLU	MAC datapath + microprogram
Decision stage	Interpret score as input to a classifier	External comparator or next layer

3. Overall Architecture

The design is divided into a microprogram control path and a neural-computation datapath. The MicroROM provides the current microinstruction; the decoder interprets its fields; the microsequencer chooses the next microaddress; and the datapath performs the requested arithmetic and register transfers.

Block	Responsibility	Concept represented
MicroROM / control store	Stores the 11-bit microinstructions	Microprogram
Microsequencer	Updates µPC; handles sequential flow, branch, halt	µPC and sequencing
Microcode decoder	Decodes opcode and next-address fields	Vertical microcode
Activation buffer	Stores input vector X as signed INT8 values	Activations / vector
Weight buffer	Stores weight vector W as signed INT8 values	Learned weights
MAC unit + accumulator	Computes and accumulates one product per iteration	Dot product
Bias + ReLU + output	Completes the neuron calculation and stores output	Bias / activation / output
Top-level module	Connects interfaces and submodules	System integration

Numerical example (vector length N = 4)

x = [2, 3, 1, 4] w = [1, 2, 3, 1] b = 2

Dot product = 2×1 + 3×2 + 1×3 + 4×1 = 15; add bias = 17; ReLU(17) = 17.

Numeric formats: activation = signed INT8; weight = signed INT8; product = signed INT16; accumulator, bias, and output = signed INT32. The implementation must use explicit signed declarations and appropriately sized arithmetic to avoid unintended unsigned operations or truncation.

4. Microprogram Flow

Address	Operation	Register-transfer intent
µ0	INIT	ACC ← 0; addresses ← 0; COUNT ← N; capture bias
µ1	MAC	ACC ← ACC + X[XADDR] × W[WADDR]
µ2	LOOP	Decrement count; advance addresses if another element remains; branch to µ1 or proceed to µ3
µ3	ADD_BIAS	ACC ← ACC + BIAS
µ4	RELU	ACC ← max(0, ACC)
µ5	WRITE_OUT	OUT ← ACC; indicate result ready
µ6	HALT	Remain idle until next accepted start

5. Microinstruction Format

The initial vertical microinstruction is 11 bits wide: opcode[3:0], next_mode[1:0], target[4:0]. The opcode names the datapath operation; next_mode selects control flow; target provides a branch address.

Bits	Field	Purpose
[10:7]	Opcode (4 bits)	Selects INIT, MAC, LOOP, ADD_BIAS, RELU, WRITE_OUT, HALT
[6:5]	Next mode (2 bits)	00 = sequential; 01 = unconditional jump; 10 = conditional loop; 11 = reserved
[4:0]	Target (5 bits)	Branch destination; supports addresses 0–31

Sequencing rules

• INIT → µ1; MAC → µ2.

• At LOOP, if more elements remain, branch to µ1; otherwise continue to µ3.

• ADD_BIAS → µ4 → µ5 → µ6; HALT remains at µ6 until a new computation is accepted.

• Use a consistent count convention. A simple option is to test whether the pre-edge count is greater than one after each MAC; decrement count and advance addresses when appropriate.

Timing note: the first simulation can use small register-array buffers with combinational reads. A physical synchronous SRAM generally has read latency, so a later implementation may need explicit read/wait micro-operations.

6. RTL Module Inventory

File	Role	Why it exists
microrom.sv	Control store	Implements stored microinstructions
microsequencer.sv	µPC and branch logic	Controls the microprogram flow
microcode_decoder.sv	Field decoder	Translates encoded fields into control signals
activation_mem.sv	Activation buffer	Stores vector X
weight_mem.sv	Weight buffer	Stores vector W
mac_unit.sv	Multiply-accumulate arithmetic	Performs signed multiply and accumulation
npu_datapath.sv	Datapath integration	Connects memories, counter, accumulator, bias, ReLU and output
npu_top.sv	Top-level system	Connects control, datapath, and external interface
npu_tb.sv	Testbench	Loads test vectors and checks output

7. GitHub Repository Structure

```
microROM-NPU/
■■■ rtl/
■   ■■■ microrom.sv
■   ■■■ microsequencer.sv
■   ■■■ microcode_decoder.sv
■   ■■■ activation_mem.sv
■   ■■■ weight_mem.sv
■   ■■■ mac_unit.sv
■   ■■■ npu_datapath.sv
■   ■■■ npu_top.sv
■■■ tb/
■   ■■■ npu_tb.sv
■■■ docs/
■   ■■■ purpose_and_applications.md
■   ■■■ architecture.md
■   ■■■ microinstruction_format.md
■   ■■■ verification_results.md
■■■ README.md
■■■ LICENSE
```

8. Verification Plan

Test	Purpose	Expected check
Positive values	Basic arithmetic path	Output equals software reference
Negative weights / inputs	Signed arithmetic correctness	Correct signed product and sum
All-zero vector	Zero-product behavior	Output determined by bias and ReLU
Negative final sum	ReLU behavior	Output is zero
N = 1, 4, and maximum supported N	Loop boundaries	Exactly N MAC operations; no out-of-range reads
Reset / repeated starts	Control protocol	No stale accumulator or corrupted in-flight operation

9. Scope and Limitations

• This is a single-neuron sequential-MAC inference kernel, not a complete commercial NPU.

• It does not train a model, execute an LLM, include DMA/AXI, or implement a systolic array.

• A real application needs trained weights, a defined feature-scaling method, and evaluation on representative data.

• Performance claims should be based on measured simulation cycles and synthesis results, not assumed TOPS or power figures.

Recommended implementation order: verify the MicroROM encoding first, then implement and test the microsequencer, then the datapath, then integrate the top-level interface and run end-to-end reference-model tests.